FIT5225 - Cloud Computing for Big Data

Assignment 1 - Benchmark Report

CloudEco - Wildfire & Smoke Detection on Kubernetes

Surya Ganesh

Student ID: 35415940

Model 1 - Wildfire & Smoke Detection (Fire, Smoke)

Public endpoint: http://169.224.230.182:30080

1. Results Table

The cluster (1 master + 2 workers, 4 vCPU / 8 GB each) was tested at 1, 2, 4 and 6 pods, each capped at 1 vCPU. 6 pods was the maximum schedulable configuration given available worker node memory. For each pod count, users were ramped until latency degraded sharply or HTTP errors appeared; the last stable point is reported.

Pods	Max stable users	Avg response	RPS	Failures
1	5	3044.97 ms	0.94	0%
2	10	2604.00 ms	1.84	0%
4	20	3546.12 ms	3.21	0%
6	20	1668.26 ms	5.02	0%

Table 1. Stable-point users, latency and throughput per pod count.

2. Graphical Plots

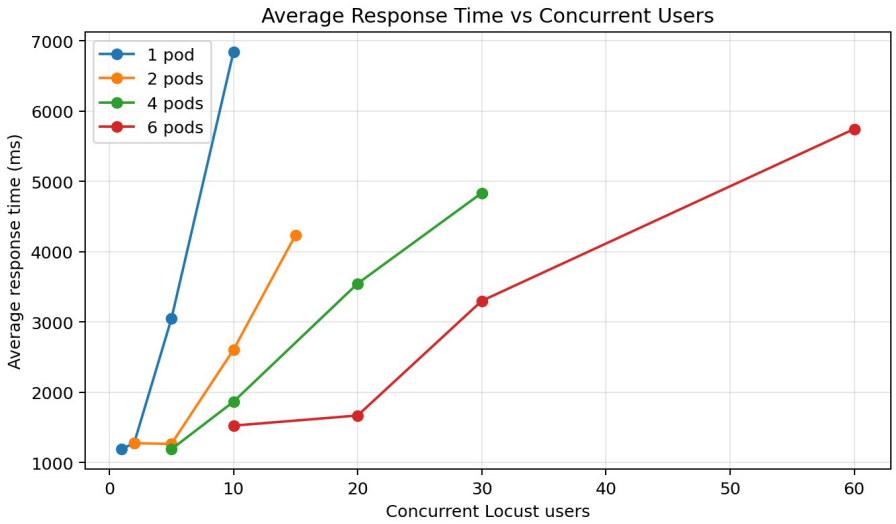

Figure 1. Avg response time vs concurrent users for 1, 2, 4 and 6 pods.

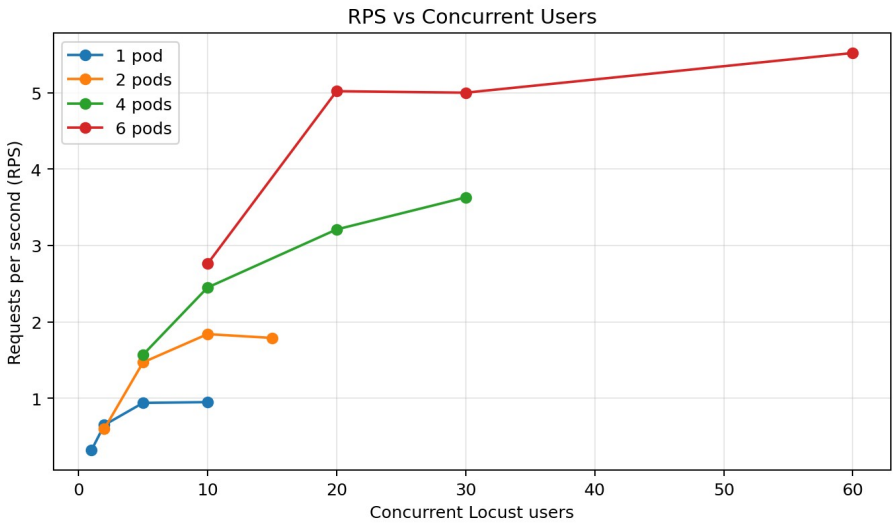

Figure 2. RPS vs concurrent users for 1, 2, 4 and 6 pods.

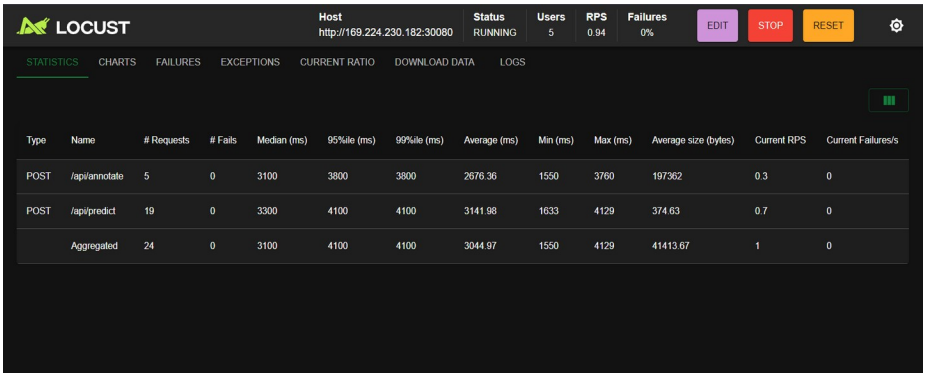

Figure 3. 1 pod / 5 users - 3044.97 ms avg, 0.94 RPS, 0 failures.

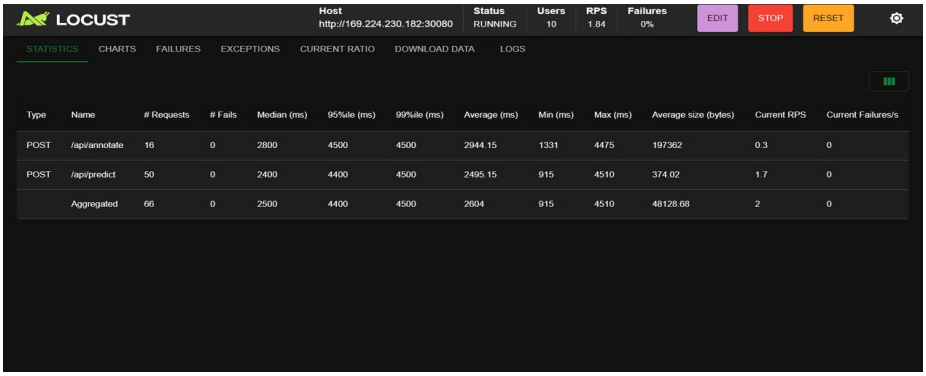

Figure 4. 2 pods / 10 users - 2604.00 ms avg, 1.84 RPS, 0 failures.

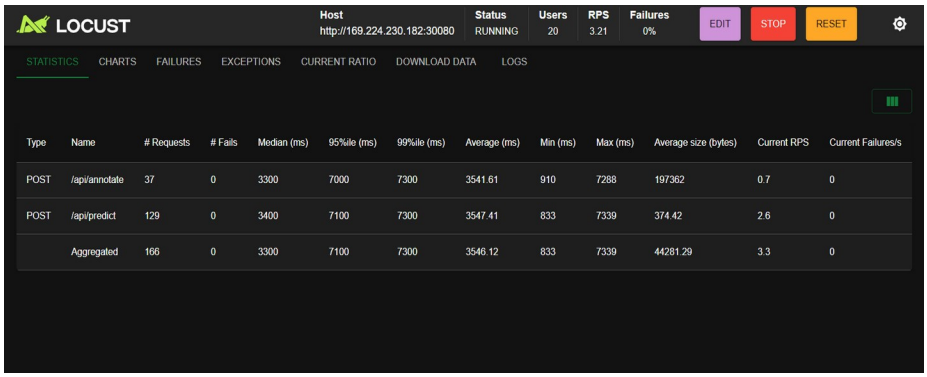

Figure 5. 4 pods / 20 users - 3546.12 ms avg, 3.21 RPS, 0 failures.

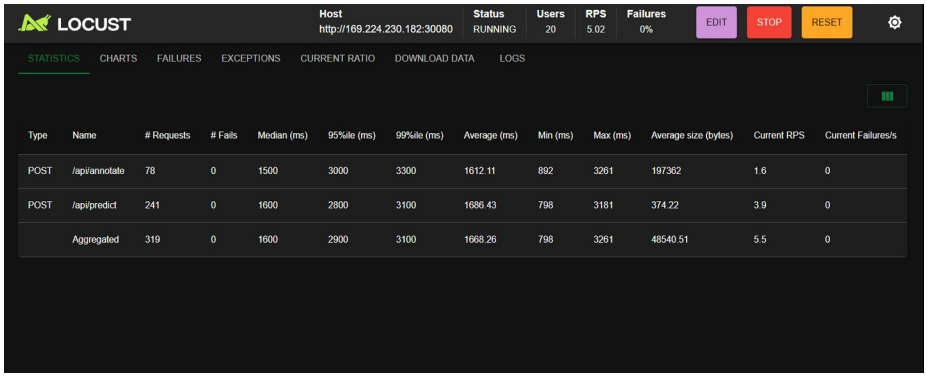

Figure 6. 6 pods / 20 users - 1668.26 ms avg, 5.02 RPS (best stable point).

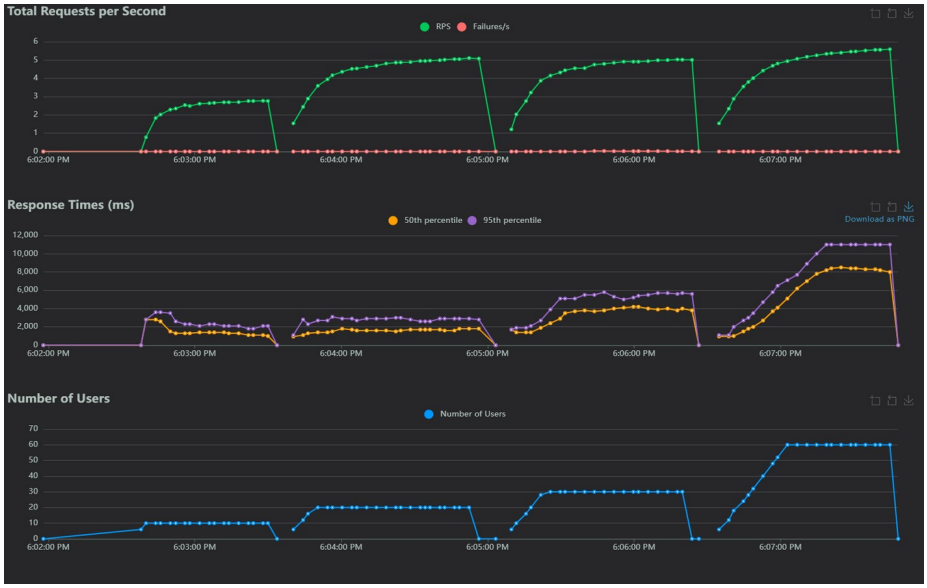

Figure 7. 6-pod live charts across the 10, 20, 30, 60 user steps.

3. Performance Analysis

With 1 pod the system saturates at 5 concurrent users: pushing to 10 users more than doubles latency (3.04 s to 6.84 s) while RPS plateaus at 0.95. By Little's Law ($L = \lambda W$), when arrival rate exceeds the per-pod service rate, requests queue inside the FastAPI thread pool and W grows with queue depth. YOLO

inference on 1 vCPU takes ~1.0–1.1 s per image, giving a hard service ceiling near 1 req/s; any higher arrival rate drives the M/M/1 curve into its steep region, exactly the spike seen in Figure 1.

Adding pods adds independent service channels. Stable RPS scales near-linearly (0.94 → 1.84 → 3.21 → 5.02), confirming horizontal scaling is the correct mitigation for this CPU-bound workload. The slope flattens between 4 and 6 pods as pods contend for shared L3 cache and kernel scheduling on each worker, so speed-up is no longer perfectly linear.

The dominant bottleneck is YOLO inference. Ultralytics speed_inference_ms returned 850–980 ms per request while preprocess and postprocess together stayed under 80 ms. Async code alone cannot raise throughput—it only prevents event-loop blocking. Real throughput gains require more cores, a quantised model (e.g. YOLOv8n int8), or batched inference.

Horizontal Pod Autoscaling at 70% CPU target would scale replicas automatically as load grows, keeping each pod in the linear region of Figure 1. The trade-off is YOLO cold-start cost (~6 s), so a minimum of 2 warm replicas should be maintained to absorb sudden spikes without dropped requests.

Generative AI Acknowledgement

AI assistance was used to debug a thread-pool config and sanity-check Locust statistics. All prompts and outputs are submitted in the separate prompts PDF as required.